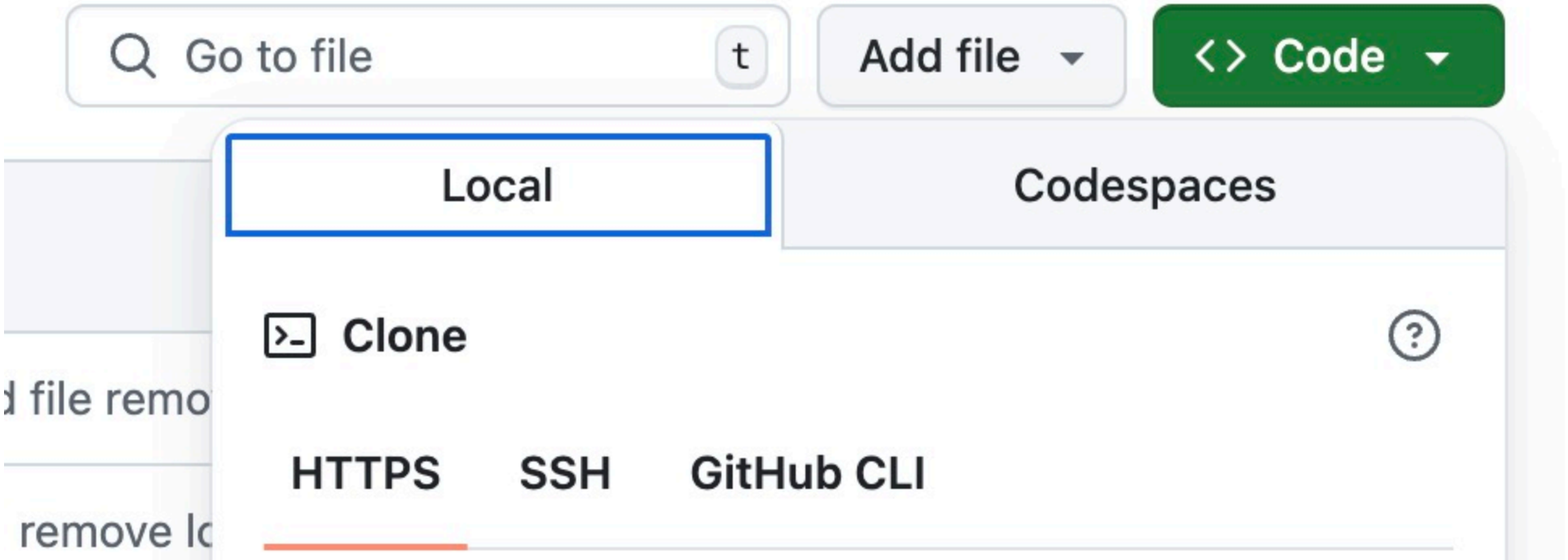
0703 分享

前置作業

- 下載 vs code
- 下載 android studio
- 下載倉庫



ToC

一開始，要先下載模擬器

1. 功能與技術對應介紹：服務藍圖
2. 前端介面程式碼簡介：單一資料流、MVVM
3. 後端伺服器程式碼簡介：prompt 設計

Frontend MVVM & Single Flow 設計原則

🏗️ MVVM 架構概述

MVVM (Model-View-ViewModel) 是一種將 UI 邏輯與業務邏輯分離的架構模式，特別適合 Android 開發。

核心組件

```
View (UI層) ↔️ ViewModel (邏輯層) ↔️ Model (資料層)
```

- **View**: Fragment/Activity，負責 UI 顯示
- **ViewModel**: 處理 UI 邏輯，管理 LiveData
- **Model**: Repository/DataSource，處理資料存取

🔄 Single Flow 設計原則

核心概念：單一資料流

關鍵優勢：

- 資料流向單一且可預測
- 狀態管理集中化
- 便於除錯和測試
- 避免資料不一致

實際應用

```
User Action → ViewModel → Repository → Server → Repository → ViewModel → View
```

- 使用者：說一句話
- ViewModel: 因為語音轉文字（STT）辨識完成，從 listening 切換到 thinking，並且通知 Repository
- Repository: 將辨識完的語句傳到伺服器
- Server: 將辨識完的語句送到 OpenAI 生成回應
- Repository: 收到伺服器的回應，從 thinking 切換到 speaking，
- ViewModel: 進行文字轉語音（TTS），語音播放完成後，從 speaking， 切換到 listening

動作與表情設計

(a) 說話動作

(b) 聆聽動作

(c) 思考動作

(d) 說話表情

(e) 聆聽表情

(f) 思考表情

📱 本專案的 MVVM 實作

1. View Layer - VideoFragment

```java
// VideoFragment.java - UI 層
public class VideoFragment extends Fragment {
    private RobotViewModel robotViewModel;
    private SharedViewModel sharedViewModel;
    private VideoView videoView;

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        // 取得 ViewModel 實例
        robotViewModel = new ViewModelProvider(requireActivity()).get(RobotViewModel.class);
        sharedViewModel = new ViewModelProvider(requireActivity()).get(SharedViewModel.class);

        // 觀察資料變化
        sharedViewModel.getEmotionVideoMap().observe(this, map -> {
            this.emotionVideoMap = map;
        });
    }

    private void observeEmotionChanges() {
        // 觀察情緒變化，更新 UI
        robotViewModel.getEmotionLiveData().observe(getViewLifecycleOwner(), emotionKey -> {
            if (emotionKey != null) {
                String videoPath = emotionVideoMap.get(emotionKey);
                if (videoPath != null) {
                    Uri videoUri = Uri.parse(videoPath);
                    videoView.setVideoURI(videoUri);
                    videoView.start();
                }
            }
        });
    }
}
```

2. ViewModel Layer - RobotViewModel

```java
// RobotViewModel.java - 邏輯層
public class RobotViewModel extends ViewModel {
    private final DataRepository dataRepository;
    private final MutableLiveData<String> emotionLiveData = new MutableLiveData<>();
    private final MutableLiveData<Boolean> ttsPlayingLiveData = new MutableLiveData<>();

    public RobotViewModel(NuwaRobotAPI robotAPI, DataRepository dataRepository, CameraHandler cameraHandler) {
        this.dataRepository = dataRepository;

        // 觀察 Repository 的狀態變化
        LiveData<StatusUpdate> statusLiveData = dataRepository.getStatusLiveData();
        statusLiveData.observeForever(statusUpdate -> {
            if (statusUpdate != null) {
                setStatus(statusUpdate);  // 處理狀態變化
            }
        });
    }

    // 狀態處理 - Single Flow 的核心
    public void setStatus(StatusUpdate statusUpdate) {
        String fullEmotionKey = getFullEmotionKey(statusUpdate.getStatus());
        emotionLiveData.setValue(fullEmotionKey);  // 更新 UI

        switch (statusUpdate.getStatus()) {
            case "thinking":
                dataRepository.sendDataViaHttp(statusUpdate.getResultString(), "");
                break;
            case "speaking":
                ttsPlayingLiveData.setValue(true);
                mRobotAPI.startTTS(statusUpdate.getResultString());
                break;
            case "listening":
                ttsPlayingLiveData.setValue(false);
                mRobotAPI.startMixUnderstand();
                break;
        }
    }

    // 提供 LiveData 給 View 觀察
    public LiveData<String> getEmotionLiveData() {
        return emotionLiveData;
    }
}
```

🎯 Single Flow 的實際運作

完整的資料流程

```
1. User 說話 → Voice Event Listener
2. Voice Listener → DataRepository.updateStatus("thinking")
3. DataRepository → RobotViewModel.setStatus()
4. RobotViewModel → HTTP 請求到後端
5. 後端回應 → DataRepository.updateStatus("speaking")
6. RobotViewModel → 更新 emotionLiveData
7. VideoFragment → 觀察到變化，更新 UI
```

關鍵實作：事件監聽器

```java
// CustomVoiceEventListener.java
public class CustomVoiceEventListener implements VoiceEventListener {
    private final DataRepository dataRepository;

    @Override
    public void onMixUnderstandComplete(boolean isError, ResultType resultType, String s) {
        if (!isError && !result_string.trim().isEmpty()) {
            // 觸發 Single Flow
            dataRepository.updateStatus(new StatusUpdate("thinking", result_string));
        }
    }

    @Override
    public void onTTSComplete(boolean isError) {
        if (!isError) {
            // TTS 完成，回到聆聽狀態
            dataRepository.updateStatus(new StatusUpdate("listening"));
        }
    }
}
```

✅ 設計優勢

1. 狀態管理集中化

- 所有狀態變化都經過 `DataRepository.updateStatus()`
- 單一真實來源 (Single Source of Truth)

2. UI 與邏輯分離

- Fragment 只負責 UI 更新
- ViewModel 處理業務邏輯
- Repository 管理資料存取

3. 可測試性

- 每層職責清楚，便於單元測試
- LiveData 機制便於模擬和驗證

4. **生命週期安全**

- LiveData 自動處理生命週期
- 避免 memory leak 和 crash

5. **可擴展性**

- 新增功能只需修改對應層級
- 不影響其他組件

🔧 Factory Pattern 的使用

```java
// RobotViewModelFactory.java
public class RobotViewModelFactory implements ViewModelProvider.Factory {
    private final NuwaRobotAPI mRobotAPI;
    private final DataRepository dataRepository;
    private final CameraHandler cameraHandler;

    @NonNull
    @Override
    public <T extends ViewModel> T create(@NonNull Class<T> modelClass) {
        if (modelClass.isAssignableFrom(RobotViewModel.class))
            return (T) new RobotViewModel(mRobotAPI, dataRepository, cameraHandler);
        throw new IllegalArgumentException("Unknown ViewModel class");
    }
}
```

這種設計確保了：

- **依賴注入**：ViewModel 獲得所需的依賴
- **生命週期管理**：ViewModel 正確地與 Activity 生命週期綁定
- **資源管理**：統一管理 API 和 Handler 實例

後端 LLM Chatbot 設計架構

🏛️ 整體架構概述

```
Flask Server (Python) → OpenAI GPT-4 API → 教育問答系統
```

核心特色

- **教育導向**：專為磯永吉小屋展覽設計的問答系統
- **階段式互動**：知識問答 + 體驗收集的兩階段設計
- **狀態管理**：追蹤對話進度和使用者回應
- **個人化回饋**：根據答案正確性提供不同回應

🗂️ 資料結構設計

對話資料格式

```
# 對話記錄結構
{
  "turns": 7,
  "conversation": [
    {
      "timestamp": "2025-04-11 12:01:34",
      "speaker": "user",
      "message": "hi"
    },
    {
      "timestamp": "2025-04-11 12:01:38",
      "speaker": "bot",
      "message": "你好，我是阿蓬，歡迎參觀磯永吉小屋..."
    }
  ],
  "is_ended": false
}
```

🧠 核心邏輯實作

1. 主要路由處理

```python
# app.py - 核心聊天邏輯
@app.route('/api/chat', methods=['POST'])
def chat():
    data = request.json
    user_message = data.get('message', '')
    user_name = data.get('user_name')

    # 儲存使用者訊息
    save_message(user_name, "user", user_message)

    # 獲取機器人回覆
    response = jxw_bot(user_name, user_message)

    # 儲存機器人回覆
    save_message(user_name, "bot", response["question"])

    return jsonify(response)
```

2. 階段式對話設計

```
def jxw_bot(user_name, question):
    user_data = get_user_data(user_name)
    turns = user_data.get("turns", 0)

    # 第一階段：知識問答 (turns 0-2)
    if turns <= 2:
        system_prompt = system_prompt_template.format(
            intro=intro_text,
            context=history,
            turns=turns
        )
    # 第二階段：體驗訪談 (turns 3-6)
    elif turns > 2 and turns <= 6:
        system_prompt = interview_prompt.format(
            context=history,
            turns=turns
        )
    # 結束階段 (turns > 8)
    elif turns > 8:
        return {
            "question": "謝謝你們今天來參觀！期待下次再見！",
            "is_ended": True
        }
```

3. 知識問答階段 Prompt 設計

```
system_prompt_template = """
你是一名在台大磯永吉小屋工作的機器人阿蓬，負責與參觀完畢的同學互動，
你要從文本{intro}中，提出三個簡單的問題、與同學進行問答互動，
幫助同學回顧當天參展的經驗，你們目前已經聊了這些：{context}。

你需要遵循以下問答流程：
1. 首先自我介紹，並且說明接下來的互動，然後提出一個簡單問題
2. 等待同學回答
3. 根據同學的回答進行評估：
 - 如果答對了，確認他們的答案並給予鼓勵
 - 如果答錯了或不知道，提供正確答案並簡短解釋
 - 如果答案部分正確，肯定正確部分並補充完整答案
4. 然後再提出下一個問題

你要用繁體中文回答問題，保持禮貌並使用簡單的語言，讓同學感到輕鬆愉快。一句回應只應包含一個問題。

回答的格式為 json 格式，並包含：
1. question(string) - 你對同學的回應或問題
2. is_ended(bool) - 對話是否結束
"""
```

4. 體驗訪談階段 Prompt 設計

```
interview_prompt = """
你是一名在台大磯永吉小屋工作的機器人阿蓬，負責與參觀完畢的同學互動，
你們目前已經聊了這些：{context}，按照時間順序，回答使用者最新的一個回答。

依序詢問下列三個問題，根據使用者的回應{context}給予適切的回答：
1. 詢問小組今天參觀中印象最深刻的事情
2. 詢問小組會想要再來參觀？或推薦家人朋友來參觀嗎？
3. 詢問小組是否有其他問題，或是對於展覽的建議嗎？
4. 結束對話：謝謝你今天來參觀。

你要用繁體中文回答問題，保持禮貌並使用簡單的語言，讓同學感到輕鬆愉快。一句回應只應包含一個問題。

回答的格式為 json 格式，並包含：
1. question(string) – 你對同學的回應或問題
2. is_ended(bool) – 對話是否結束
"""
```

🔄 OpenAI API 整合

GPT-4 API 調用

```
def jxw_bot(user_name, question):
    try:
        completion = client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": "同學回應道：" + question},
            ],
            max_tokens=500,
        )

        raw = completion.choices[0].message.content

        # JSON 解析處理
        try:
            json_text = raw.replace('```json', '').replace('```', '').strip()
            data = json.loads(json_text)

            # 更新對話狀態
            if data.get("is_ended", False):
                user_data["is_ended"] = True
            elif "question" in data and "?" in data["question"]:
                user_data["turns"] += 1

            return data
        except json.JSONDecodeError:
            return {
                "question": raw,
                "is_ended": False
            }
    except Exception as e:
        return {
```

💾 狀態管理系統

使用者資料管理

```
def get_user_data(user_name):
    """獲取用戶資料，如果不存在則初始化"""
    file_path = f"data/conversations/{user_name}.json"
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        user_data = {
            "turns": 0,
            "conversation": [],
            "is_ended": False
        }
        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(user_data, f, ensure_ascii=False, indent=2)
        return user_data

def save_message(user_name, speaker, message):
    """儲存訊息並更新用戶資料"""
    user_data = get_user_data(user_name)
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    user_data["conversation"].append({
        "timestamp": timestamp,
        "speaker": speaker,
        "message": message
    })

    file_path = f"data/conversations/{user_name}.json"
```

🎯 設計亮點

1. 階段式互動設計

- **知識測驗階段**：確保學習效果
- **體驗收集階段**：收集使用者回饋
- **自然轉換**：seamless 的階段轉換

2. 智慧回應機制

```
# 根據答案正確性提供不同回應
if answer_correct:
    response = "答對了！" + explanation + next_question
else:
    response = "其實是" + correct_answer + explanation + next_question
```

3. 錯誤處理

```python
# 多層錯誤處理
try:
    # OpenAI API 調用
    completion = client.chat.completions.create(...)
except Exception as e:
    return fallback_response

try:
    # JSON 解析
    data = json.loads(response)
except json.JSONDecodeError:
    return {"question": raw_text, "is_ended": False}
```

4. 資料持久化

- 每次對話都儲存至 JSON 檔案
- 支援對話中斷後恢復
- 便於後續分析使用者行為

🔧 API 設計

RESTful 接口

```
# 主要聊天接口
POST /api/chat
{
    "message": "使用者訊息",
    "user_name": "使用者識別"
}

# 重置對話
POST /api/reset
{
    "user_name": "使用者識別"
}

# 創建新使用者
POST /api/create_user
# Returns: {"user_name": "uuid"}
```

回應格式

```
{
    "question": "機器人回應內容",
    "is_ended": false,
    "emotion": "neutral"  # 未來可擴充情緒
}
```

📊 實際對話範例

根據 `conversations/6.json` 的真實對話：

```
User: "hi"
Bot: "你好，我是阿蓬，歡迎參觀磯永吉小屋的同學們。今天我們做個簡單的互動小問答，首先請問磯永吉小屋是在哪一年建立的呢？"

User: "25"
Bot: "不錯哦！磯永吉小屋確實是在1925年建立的。接下來，我們來回顧一下，磯永吉教授與哪一位合作培育出蓬萊米呢？"

User: "沒有人"
Bot: "其實是有的哦！磯永吉教授是與末永仁合作培育出蓬萊米的。這對臺灣的農業有很大的貢獻呢。最後一個問題，磯永吉小屋是在2009年被哪個政府機構指定為市定古蹟的呢？"
```

這展現了系統的：

- **智慧判斷**：理解"25"指的是1925年
- **錯誤糾正**：溫和地提供正確答案
- **流程控制**：按序進行三個問題